DIY 1 — Modeling a stream of political apologies with a relational event model

remverse hands-on: manual risk sets, memory, and diagnostics

Relational Event Modeling workshop

2026-07-29

> **How to use this document.** Every code block is ready to paste into an R session and run in order. To keep the handout light the chunks are *not* executed while knitting (`eval = FALSE`); the expected results are described in the text. To run it live, flip `eval = FALSE` to `eval = TRUE` or copy the blocks into R. The whole analysis runs in well under a minute.

Introduction and learning objectives

This is the **gentle** of the two hands-on analyses. The data are small — 367 official **apologies** — but the *structure* is rich, which makes them the perfect setting to learn the modeling decision that trips people up most: **how to define the risk set**.

A directed event here is: a **government** publicly apologizes to a **recipient** at a given date. Recipients come in three kinds:

- another **state** (`class == "international"`, e.g. Japan → South Korea),
- a **domestic minority** of the sender's own country (`class == "domestic"`, e.g. Canada → its Indigenous peoples), or
- a cross-national **collective** (`class == "multilateral"`, e.g. Germany → Holocaust victims).

By the end you will be able to:

- read a heterogeneous event set and see why the standard risk sets fail;
- **build a manual risk set** from structural possibility;
- attach an **event weight** (the contrition ladder);
- fit a tie-oriented REM with `remstats` + `remstimate` (MLE);
- **optimize the memory half-life from the data** via BIC and recall; and
- read `diagnostics()` — recall, and the *surprise offenders* that point to what the model is missing.

Setup: packages and data

```
# install.packages(c("remify", "remstats", "remstimate"))
library("remify")      # represent the relational event history
library("remstats")    # compute endogenous / exogenous statistics
library("remstimate")  # estimate the REM (MLE)
```

```
# Point this at the workshop_data/apologies folder.
data_dir <- "workshop_data/apologies"
```

```r
edgelist     <- read.csv(file.path(data_dir, "apologies_rem.csv"),
                         stringsAsFactors = FALSE)
# actor level attributes
country_info <- read.csv(file.path(data_dir, "country_info.csv"),
                         stringsAsFactors = FALSE)
# dyadic level attributes
colonial_ties <- read.csv(file.path(data_dir, "colonial_ties.csv"),
                          stringsAsFactors = FALSE)

edgelist$date <- as.Date(edgelist$date)
edgelist      <- edgelist[order(edgelist$date), ]
head(edgelist[, c("date", "sender_id", "receiver_id", "class", "contrition")])
```

Actor identifiers follow a strict convention that we will exploit:

- senders and international receivers end in `_gov` (e.g. `"Japan_gov"`);
- domestic receivers end in `_min` (e.g. `"Canada_min"`);
- multilateral receivers are named collectives (e.g. `"holocaust_victims"`).

Get to know the data

```r
nrow(edgelist)                      # 367 events
table(edgelist$class)               # domestic / international / multilateral
table(edgelist$contrition)          # 1_acknowledge < 2_regret < 3_apology < 4_apology_redress
sort(table(edgelist$sender_id), decreasing = TRUE)[1:6]   # Japan, Germany, UK, ...
```

Two categorical columns will do real work below:

- **`class`** — this is *not* a covariate; it tells us which dyads are *structurally possible*, and therefore drives the **risk set**.
- **`contrition`** — an ordinal *intensity* ladder we will use as an **event weight**.

Choice of the risk set

At every event the REM compares the observed apology against all dyads *at risk*. Which dyads are "at risk" is a modeling choice. In **`remverse`**, there are three default choices:

- The **`full`** risk set assumes that all actors are potentially at risk to make a formal apology towards all other actors. When would this choice be reasonable, and when not?
- The **`active`** riskset keeps only dyads **observed at least once**. This is the risk set is minimally sized risk set. However it assumes that apologies that have not observed are treated as **not at risk at all** — which inflates endogenous effects like inertia (the model never "sees" the roads not taken). When would this choice be reasonable, and when not?
- The **`active_saturated`** riskset keeps only dyads that are **observed at least once** in either one of the directions. So if only A → B is observed and B → A is not, then this option could still include B → A in the riskset. When would this choice be reasonable, and when not?

  The principle. The risk set is the set of dyads that *could* produce the next event. Choose it by **structural possibility, not probability**. Keep every dyad that *could* occur; drop only the *impossible*; let improbable-but-possible pairs stay in and be absorbed by the baseline.

When events are formal apologies between countries, argue whether one of the above options are realistic. If none are realistic, a **manual** risk set is required.

Building the manual risk set

Three blocks of possible dyads, mirroring the three event classes:

```
# --- actor inventory ------------------------------------------------------
# senders are always governments; international receivers are governments too.
govs      <- sort(unique(c(edgelist$sender_id,
                   edgelist$receiver_id[edgelist$class == "international"])))
countries <- sub("_gov$", "", govs)                        # "Japan_gov" -> "Japan"
colls     <- sort(unique(edgelist$receiver_id[edgelist$class == "multilateral"]))

# --- (a) international: gov -> gov, structurally plausible pairs only ----------
# Plausible = same region, OR a colonial relationship.
reg   <- read.csv(file.path(data_dir, "region_map.csv"), stringsAsFactors = FALSE)
region_of <- setNames(reg$region, paste0(reg$country, "_gov"))

ties  <- read.csv(file.path(data_dir, "colonial_ties.csv"), stringsAsFactors = FALSE)
ties  <- ties[ties$colonial == 1, c("actor1", "actor2")]
colonial_pairs <- c(paste(ties$actor1, ties$actor2),
                    paste(ties$actor2, ties$actor1))    # symmetric: possibility is undirected

gg <- expand.grid(actor1 = govs, actor2 = govs,
                  KEEP.OUT.ATTRS = FALSE, stringsAsFactors = FALSE)
gg <- gg[gg$actor1 != gg$actor2, ]
same_region <- region_of[gg$actor1] == region_of[gg$actor2]
is_colonial <- paste(gg$actor1, gg$actor2) %in% colonial_pairs
gg <- gg[same_region | is_colonial, ]

# --- (b) domestic: each government -> its OWN minority node only --------------
gm <- data.frame(actor1 = govs,
                 actor2 = paste0(countries, "_min"),
                 stringsAsFactors = FALSE)

# --- (c) multilateral: each government -> each collective node ----------------
gc <- expand.grid(actor1 = govs, actor2 = colls,
                  KEEP.OUT.ATTRS = FALSE, stringsAsFactors = FALSE)

riskset_manual <- unique(rbind(gg, gm, gc))
rownames(riskset_manual) <- NULL
cat("risk-set dyads:", nrow(riskset_manual),
    "| govs:", length(govs), "| collectives:", length(colls), "\n")
```

Note that every observed dyad must be inside the manual risk set. This is automatically checked in the first modeling step using the `remify()` function. If events were observed for dyads that were not at risk, these dyads will be automatically added.

Actor attributes and an event weight

The **contrition ladder** could be viewed as an event weight: a fuller apology (redress) contributes more to the endogenous statistics than a bare acknowledgement.

```r
# "1_acknowledge" -> 1, "2_regret" -> 2, "3_apology" -> 3, "4_apology_redress" -> 4
edgelist$contri_weight <- as.numeric(substr(edgelist$contrition, 1, 1))
table(edgelist$contri_weight)
head(edgelist[,-10])
```

> **Type or weight?** `contrition` could also have *typed* the events (a separate process per level). We treat it as a **weight** because the substantive claim is that apologies differ in *intensity* within one apologizing process — not that redress is a different kind of act. This choice affect the way statistics are computed and how the process is modeled.

Fit the model: the four-step workflow

Step 1 — the relational event history

```r
reh <- remify(edgelist, model = "tie", riskset = "manual",
              manual_riskset = riskset_manual, time_units = "year",
              event_weight = "contri_weight")
reh
plot(reh)      # a quick descriptive of the apology stream
```

Step 2 — choose effects and compute statistics

Endogenous **mechanisms**: do apologies cluster, get reciprocated, come in bursts? (you can check their exact interpretation by running `?recencySendSender`)

```r
tie_effects1 <- ~
  inertia(scaling = "std") +                          # are apologies repeated?
  reciprocity(scaling = "std") +                      # are apologies reciprocated?
  recencySendSender()                                 # are initiators of apologies temporally active?

stats1 <- remstats(reh, tie_effects = tie_effects1, first = 50)   # 50-event burn-in
stats1
```

Additionally, it may be plausible that attributes of countries play a role as well.

```r
tie_effects2 <- ~ inertia(scaling = "std") +              # are apologies repeated?
  reciprocity(scaling = "std") +                          # are apologies reciprocated?
  recencySendSender() +                                   # are initiators of apologies temporally activ
  tie("colonial", attr_dyads = colonial_ties) +           # sender colonized receiver?
  send("western",      attr_actors = country_info) +      # do "Western" states apologize more?
  send("major_power", attr_actors = country_info)         # do major powers apologize more?

stats2 <- remstats(reh, tie_effects = tie_effects2, first = 50)   # 50-event burn-in
stats2
```

By constructing two different REMs having different sets of predictor statistics, we can assess which model is preferred for these data.

Step 3 — estimate (MLE) and read coefficients

We fit a REM to both sets of statistics

```r
fit1 <- remstimate(reh, stats1)
summary(fit1)
```

```
coef(fit1)

fit2 <- remstimate(reh, stats2)
summary(fit2)
coef(fit2)
```

Read each `send_*` as a log rate-multiplier: `exp(coef)` is how much that sender trait multiplies the apology rate, relative to the baseline. When checking the BIC of the two models which is preferred? Note that a lower BIC is preferred.

```
fit1$BIC
fit2$BIC
```

Step 4 — diagnostics

```
diag1 <- diagnostics(fit1, reh, stats1)
diag1$recall$summary                 # mean relative rank + proportion in the top 5%
plot(diag1)

# Which events did the model find most surprising? (the surprise_threshold is set to .2 by default)
head(diag1$surprise_offenders)
head(diag1$surprise_offenders_actor$sender)     # senders the model under-predicts
```

A cluster of surprises around specific senders, receivers, or dyads usually means a **missing covariate**.

Featured: optimizing the memory half-life from the data

Grudges and moral reckonings may **fade**. It is possible to model memory via a `decay` function which down-weights old events exponentially with a given **half-life**. Rather than guess whether this is appropriate, we can explore this by considering a grid of half-life value and let the data choose. We score each half-life by **BIC** (lower = better) and by median **recall** (higher = better).

```
half_lives <- seq(1, 16, by = 3) # on 'year' time scale

scores <- do.call(rbind, lapply(half_lives, function(h) {
  st  <- remstats(reh, tie_effects = tie_effects1,
                  memory = "decay", memory_value = h, first = 50)
  fit <- remstimate(reh, st)
  dg  <- diagnostics(fit, reh, st)
  data.frame(half_life = h,
             BIC    = fit$BIC,
             recall = dg$recall$summary[1, 2])
}))
scores
```

```
# which half-life minimizes the BIC?
h_bic <- scores$half_life[which(scores$BIC==min(scores$BIC))]
# which half-life maximizes recall?
h_recall <- scores$half_life[which(scores$recall==max(scores$recall))]
plot(scores$half_life, scores$BIC,    type = "b",
     xlab = "half-life (years)", ylab = "BIC");    abline(v = h_bic,    col = 2)
plot(scores$half_life, scores$recall, type = "b",
     xlab = "half-life (years)", ylab = "recall"); abline(v = h_recall, col = 2)
```

The chosen half-life is itself a **finding**: it says how long an apology stays relevant in future apology events. Refit at the optimum and interpret:

```
stats_opt <- remstats(reh, tie_effects = tie_effects1,
                      memory = "decay", memory_value = h_bic, first = 50)
fit_opt   <- remstimate(reh, stats_opt)
summary(fit_opt)
fit_opt$BIC
fit1$BIC
```

Exercises

Predict the outcome first, then run it.

1. **Risk-set sensitivity.** Refit with `riskset = "active"` (drop `manual_riskset`). How does this change the interpretation of the results?
2. **A different mechanism.** Add other potentially important endogenoud and/or exogenous `tie_effects` to see whether this affects the results.
3. **Weight or no weight?** Refit *without* the contrition weight (rebuild `reh` without setting `edgelist$weight`). Does weighting by intensity change the endogenous estimates?

For each, report the coefficients (as rate multipliers), model fit (BIC and/or AIC),the recall summary, and substantive interpretation.

References

Butts, C. T. (2008). A relational event framework for social action. *Sociological Methodology, 38*(1), 155–200.

Leenders, R. T. A. J., Contractor, N. S., & DeChurch, L. A. (2016). Once upon a time: Understanding team processes as relational event networks. *Organizational Psychology Review, 6*(1), 92–115.

Meijerink-Bosman, M., Back, M., Geukes, K., Leenders, R., & Mulder, J. (2023). Discovering trends of social interaction behavior over time: An introduction to relational event modeling. *Behavior Research Methods, 55*, 997–1023.